

claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_56274ff0-603\agent-a090624ce3407f05e.jsonl`  
- SHA-256: `c4a134d840890d8646430253fea21271406b767bbe90719c1591e2c8337d3b56`  
- Source modified: `2026-06-21T02:36:18+00:00`  
- Imported at: `2026-07-05T16:48:22+00:00`  
- project: `wf_56274ff0-603`  
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`
```

Transcript

1. user (2026-06-21T02:34:25.856Z)

Map the CORE rally engine ? detection + stitching ? that MUST stay identical across deployment modes. Read backend/pipeline/segmenters/trajectory_hybrid.py (the infer flow + _resolve_runner detector seam), backend/pipeline/detectors/{native_wasb_runner,wasb_runner,stub_runner,base}.py, and ESPECIALLY backend/pipeline/stitcher/compiler.py (VideoCompiler.compile_highlights, extract_video_chunk, the ffmpeg per-clip re-encode + watermark + concat). Identify: (1) the CORE CONTRACT ? the pure function "(video bytes + segments + config) -> rallies / reel" that must be behaviour-identical local vs cloud; (2) exactly WHERE GPU compute (detect) and ffmpeg compute (stitch) happen; (3) every COUPLING POINT to the local filesystem / hardcoded paths / device assumptions that would differ across local-vs-cloud. Tag each finding kind + reuse_status.

2. assistant (2026-06-21T02:34:27.746Z)

I'll map the core rally engine components systematically. Let me start by exploring the codebase structure and then read the critical files.

3. user (2026-06-21T02:34:28.602Z)

```
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/native_wasb_runner.py  
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/stub_runner.py  
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/tracknet_runner.py  
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py  
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/trajectory_hybrid.py  
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py
```

4. assistant (2026-06-21T02:34:30.371Z)

Now let me read the key files to map the core engine.

5. user (2026-06-21T02:34:31.027Z)

```
1      """Shared engine for trajectory-substrate hybrid segmenters.  
2  
3      tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a  
4      standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is  
5      now the single home for the whole pipeline shape:  
6  
7          healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)  
8          -> P3 AI-provider handoff over padded clips -> P4 DB population/linking  
9  
10     A concrete subclass supplies only its identity and runner wiring:
```

```

11     - ``family`` ? config section under ``indexing.*``, output subdir, and
12                   the ``<family>-hybrid-<model>`` detector tag in metadata.
13     - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14     - ``name`` / ``description`` properties (the registry contract).
15     - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16                                     detection_params extras such as weights_path).
17
18     This base is deliberately NOT registered; only concrete subclasses register.
19     """
20     import os
21     import time
22     import logging
23     import concurrent.futures
24     from typing import Any, Dict, List, Optional, Tuple
25
26     import cv2
27
28     from backend.pipeline.segmenters.base import BasePlaySegmenter
29     from backend.utils.video import get_video_duration, extract_video_chunk
30     from backend.sports import sport_registry
31     from backend.providers import provider_registry
32     from backend.pipeline.detectors.base import DetectorRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56             """Return (runner, detector-specific detection_params extras) for the default
57             (WSL) path."""
58             raise NotImplementedError
59
60         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
61 Dict[str, Any]]:
62             """Return (runner, extras) for the Linux-native (no-WSL) path. Only families
63             with a
64             native runner override this; the base refuses with a clear message (M3.5: WASB
65             only)."""
66             raise NotImplementedError(
67                 f"detector_impl='native' is not supported for the "
68                 f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB
69                 has a "
70                 f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")
71
72         def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
73 Dict[str, Any]]:
74             """Resolve the detector runner, honouring the CPU-stub and Linux-native
75             overrides.

```

```

67
68         ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or
``indexing.detector_impl``:
69         ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole
pipeline
70         is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"``
swaps
71         in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
72         ``_build_native_runner``. Anything else (``"auto"``) delegates to
``_build_runner``
73         (the WSL runner) so the default production path is unchanged
74         (``docs/DECOUPLED_COMPUTE/``).
75         ""
76         impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or
"auto").lower()
77         if impl == "stub":
78             from backend.pipeline.detectors.stub_runner import StubDetectorRunner,
StubConfig
79             logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
80                         self.display_label or "trajectory")
81             return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)),
{"detector_impl": "stub"}
82         if impl in ("native", "native_wasb", "wasb_native"):
83             logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
84                         self.display_label or "trajectory")
85             return self._build_native_runner(idx_cfg)
86         return self._build_runner(idx_cfg)
87
88     @staticmethod
89     def _probe_fps_width(video_path: str) -> tuple:
90         """Return (fps, frame_width) for a video, with sensible fallbacks."""
91         cap = cv2.VideoCapture(video_path)
92         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
93         width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
94         cap.release()
95         return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
96
97     @staticmethod
98     def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
99         """Provider-reported exchange count, else a duration-based estimate.
100
101         One canonical implementation ? the twins had drifted (wasb relied on
102         ``int(None)`` raising into the fallback; same result, by accident).
103         ""
104         shot_count = segment.get("shuttle_exchange_count")
105         if shot_count is None:
106             return max(3, int(duration / 0.8))
107         try:
108             return int(shot_count)
109         except (ValueError, TypeError):
110             return max(3, int(duration / 0.8))
111
112     # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
-----
113     def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
114                                frame_width: float, video_path: str,
115                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
116         """Stats dict persisted into ``candidate_segments.stats`` for one window. The
BASE
117         returns exactly the 4 motion keys, so the trajectory twins persist byte-
identical
118         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
119         features). ``points`` are the whole-video trajectory points
(TrajectoryPoint)."""
120         return {

```

```

121         "peak_velocity": cw["peak_velocity"],
122         "mean_velocity": cw["mean_velocity"],
123         "active_frame_count": cw["active_frame_count"],
124         "track_id_count": cw["track_id_count"],
125     }
126
127     def _select_for_handoff(self, windows: List[Dict[str, Any]],
128                             window_stats: List[Dict[str, Any]],
129                             idx_cfg: Dict[str, Any]) -> List[int]:
130         """Indices of the candidate windows that proceed to the AI handoff (and thus may
131         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
132         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
133         Persisted candidates are NOT affected ? they remain the full superset for
134         offline
135         re-scoring."""
136         return list(range(len(windows)))
137
138     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
139                      player_pool: Optional[List[str]] = None,
140                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
141         # override-ignore: the ABC declares the Result contract (Success|Failure)
142         # but every live segmenter still returns a bare list ? the documented
143         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
144         label = self.display_label
145         # --- Sport profile + AI provider wiring (identical across segmenters) ---
146         sport_name = self.config.get("sport", "badminton")
147         try:
148             sport_profile = sport_registry.get(sport_name)
149         except KeyError as e:
150             logger.error(str(e))
151             return []
152
153         idx_cfg = self.config.get("indexing", {})
154         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
155         the
156         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
157         is
158         # needed and nothing is uploaded. Used to run the local detector standalone
159         (e.g. to
160         # measure it against the Gemini path, or to avoid the cloud entirely). With
161         # FusionSegmenter the windows are still Gate-A/B-filtered via
162         `_select_for_handoff`.
163         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
164         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
165         "gemini"))
166         if skip_ai:
167             model_name = "local-noai"
168             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
169         windows will "
170             "be emitted as rallies; no %s handoff, no API key needed.",
171             label, provider_name)
172         else:
173             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
174         "gemini-2.5-flash"))
175             api_key_env_var = f"{provider_name.upper()}_API_KEY"
176             api_key = os.environ.get(api_key_env_var)
177             if not api_key:
178                 from backend.pipeline.segmenters._keyhint import api_key_hint
179                 logger.error(
180                     f"{api_key_env_var} environment variable is not set! "
181                     f"To run the {provider_name} handoff, set your API key. "
182                     + api_key_hint(api_key_env_var)
183                 )
184             return []
185         try:

```

```

178         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
179     except KeyError as e:
180         logger.error(str(e))
181         return []
182
183     video_duration = get_video_duration(video_path)
184     if video_duration <= 0:
185         logger.error("Invalid video duration.")
186         return []
187     fps, frame_width = self._probe_fps_width(video_path)
188
189     # --- Runner config + windowing thresholds ---
190     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
191     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
192     # it delegates to the subclass _build_runner (the real WSL/native runner).
193     try:
194         runner, runner_params = self._resolve_runner(idx_cfg)
195     except NotImplementedError as e:
196         # e.g. detector_impl="native" for a family without a native runner
(tracknet, or
197         # fusion with a tracknet substrate). Refuse cleanly (no rallies) instead of
crashing.
198         logger.error("%s: %s", label, e)
199         return []
200
201     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
202     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
203     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
204     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
205     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
206     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
207     window_hard_cap = idx_cfg.get("window_hard_cap", False)
208     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
209     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
210     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
211     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
212     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
213     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
214     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
215     log_candidates = idx_cfg.get("log_yolo_candidates", True)
216     store_candidates = idx_cfg.get("store_yolo_candidates", True)
217
218     detection_params = {
219         "detector": self.name,
220         "velocity_thresh": velocity_thresh,
221         "merge_gap": merge_gap,
222         "min_action_window_duration": min_window_duration,
223         **runner_params,
224     }
225
226     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
227     ok, msg = runner.healthcheck()
228     if not ok:
229         logger.error("%s detector environment not ready: %s", label, msg)
230         return []
231     logger.info("%s detector env OK: %s", label, msg)
232
233     # --- Phase 2: trajectory -> candidate rally windows ---
234     # Trajectory detectors process the whole video in one pass (they carry
235     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
236     t_start = time.time()
237     out_dir = os.path.join("output", self.family)
238     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)

```

```

239         if not csv_path:
240             logger.error("%s produced no trajectory; aborting.", label)
241             return []
242
243         points = runner.parse_trajectory_csv(csv_path)
244         logger.info("Parsed %d trajectory points (%d visible).",
245                     len(points), sum(1 for p in points if p.visible))
246
247         all_candidate_windows = runner.trajectory_to_action_windows(
248             points, fps=fps, frame_width=frame_width, chunk_start=0.0,
249             velocity_thresh=velocity_thresh, merge_gap=merge_gap,
250             min_window_duration=min_window_duration, chunk_index=0,
251             max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
252             window_hard_cap=window_hard_cap,
253             window_inactivity_end=window_inactivity_end,
254             inactivity_rally_thresh=inactivity_rally_thresh,
255             inactivity_min_density=inactivity_min_density,
256             window_blob_fallback=window_blob_fallback,
257             window_blob_factor=window_blob_factor,
258         )
259         logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
260                     label, time.time() - t_start, len(all_candidate_windows))
261
262         if log_candidates:
263             for cw in all_candidate_windows:
264                 logger.info(f"[{label} rally]
265                             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
266                             f"({cw['end'] - cw['start']:.1f}s) |
267                             frames={cw['active_frame_count']} "
268                             f"peak_v={cw['peak_velocity']:.3f}
269                             mean_v={cw['mean_velocity']:.3f}")
270
271         # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
272         fusion
273         # segmenter adds net_crossings + person-cue features. Computed once, reused for
274         both
275         # persistence and the handoff gate below.
276         window_stats = [
277             self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
278             idx_cfg)
279             for cw in all_candidate_windows
280         ]
281
282         # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
283         candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
284         if store_candidates:
285             for i, cw in enumerate(all_candidate_windows):
286                 candidate_ids[i] = self.db.add_candidate_segment(
287                     video_id, detector=self.name,
288                     start_time=cw["start"], end_time=cw["end"],
289                     chunk_index=cw["chunk_index"], confidence=min(1.0,
290                     cw["peak_velocity"]),
291                     stats=window_stats[i], detection_params=detection_params,
292                 )
293
294         if not all_candidate_windows:
295             return []
296
297         # --- Phase 3: AI provider handoff over padded candidate clips ---
298         # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
299         Persisted
300         # candidates above stay the full superset ? only the served play_segments are
301         gated.
302         handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
303         idx_cfg)

```

```

294
295     ai_segments: List[dict] = []
296     if skip_ai:
297         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
298         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
299         # falls back to the duration-based estimate (no AI exchange count).
300         ai_segments = [
301             {
302                 "start_time": all_candidate_windows[wi]["start"],
303                 "end_time": all_candidate_windows[wi]["end"],
304                 "shuttle_exchange_count": None,
305                 "shot_count_confidence": "LocalNoAI",
306                 "_candidate_id": candidate_ids[wi],
307             }
308             for wi in handoff_indices
309         ]
310         logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
311                     "(no AI handoff).", len(ai_segments))
312     else:
313         handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
314         os.makedirs(handoff_dir, exist_ok=True)
315         logger.info("Phase 3: %s validation on %d candidate windows...",
316                     provider_name, len(handoff_indices))
317
318         def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
319             w_start = max(0, window["start"] - handoff_padding)
320             w_end = min(video_duration, window["end"] + handoff_padding)
321             w_dur = w_end - w_start
322             w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
323             if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
324                 return []
325             prompt = sport_profile.get_prompt(chunk_duration=w_dur)
326             segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
327                                                  label=f"Handoff {wi+1}")
328             valid = []
329             for seg in segments:
330                 try:
331                     seg["start_time"] = float(seg["start_time"]) + w_start
332                     seg["end_time"] = float(seg["end_time"]) + w_start
333                     seg["_candidate_id"] = candidate_ids[wi]
334                     valid.append(seg)
335                 except (ValueError, TypeError, KeyError) as e:
336                     logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
337             try:
338                 os.remove(w_path)
339             except OSError:
340                 pass
341             return valid
342
343         with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
344             futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
345                         for wi in handoff_indices]
346             for future in futures:
347                 ai_segments.extend(future.result())
348         try:
349             os.rmdir(handoff_dir)
350         except OSError:

```

```

351         pass
352         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
353
354         # --- Phase 4: DB population + candidate linking ---
355         segments_data = []
356         for segment in ai_segments:
357             try:
358                 start = float(segment["start_time"])
359                 end = float(segment["end_time"])
360                 dur = end - start
361                 metadata = {
362                     "shot_count": self._shot_count_from(segment, dur),
363                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
364                     "detector": f"{self.family}-hybrid-{model_name}",
365                 }
366                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
367                 if segment_id:
368                     candidate_id = segment.get("_candidate_id")
369                     if candidate_id is not None:
370                         self.db.link_candidate_to_segment(candidate_id, segment_id)
371                     segments_data.append({
372                         "id": segment_id, "start_time": start, "end_time": end,
373                         "duration": dur, "metadata": metadata,
374                     })
375             except Exception as e:
376                 logger.warning("Skipping invalid segment struct: %s", e)
377
378         return segments_data
379

```

6. user (2026-06-21T02:34:31.261Z)

```

1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)

```



```

26
27 @staticmethod
28 def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29     if watermark is None:
30         return None
31     if hasattr(watermark, "model_dump"):
32         return watermark.model_dump()
33     if isinstance(watermark, dict):
34         return dict(watermark)
35     return None
36
37 @staticmethod
38 def _ff_quote(path: str) -> str:
39     """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40     filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41     slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43     The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44     option parser treats ':' as the option separator, so a Windows drive-letter
45     path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46     the encode fails with "Invalid argument". This silently disabled the
47     on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48     return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50 @staticmethod
51 def _parse_time(val: Union[int, float, str]) -> float:
52     """Normalizes a timestamp value to float seconds.
53     Handles: float/int (pass-through), plain numeric strings ("12.4"),
54     and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55     This mirrors the parse_time logic used in gemini.py to ensure
56     the stitcher can always consume whatever format the indexer produced."""
57     if isinstance(val, (int, float)):
58         return float(val)
59     if isinstance(val, str):
60         val = val.strip()
61         if ":" in val:
62             # Handle MM:SS, HH:MM:SS, etc.
63             parts = val.split(":")
64             parts.reverse()
65             total = 0.0
66             for i, p in enumerate(parts):
67                 try:
68                     total += float(p) * (60 ** i)
69                 except ValueError:
70                     pass
71             return total
72         try:
73             return float(val)
74         except ValueError:
75             logger.warning(f"Could not parse timestamp value '{val}' as a number.
76             Defaulting to 0.0.")
77             return 0.0
78             logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
79             '{val}'. Defaulting to 0.0.")
80             return 0.0
81
82 @staticmethod
83 def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
84 float]]:
85     """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
86
87     A highlight reel must never replay the same footage. The same rally can land in
88     the segment list more than once ? e.g. two detector runs accumulated for one
89     video (add_video is an UPSERT, so re-running without a prune leaves both sets),
90     or

```

```

87         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
88         twice. Merging any range that overlaps or abuts the previous one collapses true
89         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
90         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
91         """
92         parsed = []
93         for raw_s, raw_e in segments:
94             s = VideoCompiler._parse_time(raw_s)
95             e = VideoCompiler._parse_time(raw_e)
96             if e > s:
97                 parsed.append((s, e))
98         parsed.sort()
99         merged: List[Tuple[float, float]] = []
100         for s, e in parsed:
101             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
102                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
103             else:
104                 merged.append((s, e))
105         return merged
106
107     def _get_video_duration(self, video_path: str) -> float:
108         """Probes the video file to get its total duration in seconds.
109         Returns 0.0 if it cannot be determined."""
110         try:
111             result = subprocess.run(
112                 [
113                     "ffprobe", "-v", "error",
114                     "-show_entries", "format=duration",
115                     "-of", "default=noprint_wrappers=1:nokey=1",
116                     video_path
117                 ],
118                 capture_output=True, text=True, check=True
119             )
120             return float(result.stdout.strip())
121         except Exception as e:
122             logger.warning(f"Could not determine video duration via ffprobe: {e}")
123             return 0.0
124
125     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129         because the mark is already baked into every clip identically."""
130         wm = self.watermark
131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138             logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",

```

```

149         "top-right": f"x=w-tw-{margin}:y={margin}",
150         "top-left": f"x={margin}:y={margin}",
151     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152     ov_xy = {
153         "bottom-right": f"W-w-{margin}:H-h-{margin}",
154         "bottom-left": f"{margin}:H-h-{margin}",
155         "top-right": f"W-w-{margin}:{margin}",
156         "top-left": f"{margin}:{margin}",
157     }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159     drawtext = ""
160     if text:
161         ratio = float(wm.get("font_size_ratio") or 0.035)
162         divisor = max(1, round(1.0 / ratio))
163         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
164         strings.
165         tf_path = os.path.join(tmp_dir, "watermark.txt")
166         with open(tf_path, "w", encoding="utf-8") as fh:
167             fh.write(text)
168         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
169         # without fontconfig) ? fontconfig family (last-resort, environment-
170         dependent).
171         font_path = (wm.get("font_path") or "").strip()
172         if not font_path and _BUNDLED_FONT.exists():
173             font_path = str(_BUNDLED_FONT)
174         if font_path:
175             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
176         else:
177             font_opt = f"font='{wm.get('font', 'Sans')}'"
178         box = ""
179         if wm.get("box", True):
180             box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
181         drawtext = (
182             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
183             f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
184         )
185
186     if logo_path and drawtext:
187         return
188     f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
189     if logo_path:
190         return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
191     return drawtext
192
193 def _trim_one_segment(
194     self,
195     idx: int,
196     raw_start: Union[int, float, str],
197     raw_end: Union[int, float, str],
198     segments: List[Tuple[float, float]],
199     video_duration: float,
200     tmp_dir: str,
201     raw_video_path: str,
202     watermark_filter: Optional[str],
203 ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
204     """Trim a single segment into its own clip file (extracted verbatim from the
205     per-segment body of compile_highlights). Returns (clip_path, skip_record): on
206     success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
207     None and the (idx, start, end, reason) record to record in skipped_segments."""
208     # Normalize timestamps to float seconds ? handles float, int,
209     # plain numeric strings, and colon-separated formats like "MM:SS"
210     start = self._parse_time(raw_start)
211     end = self._parse_time(raw_end)
212     segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"

```

```

211         # Validate the segment times
212         if start < 0:
213             logger.warning(f"{segment_label}: start_time is negative ({start:.2f}s).
Clamping to 0.")
214             start = 0.0
215
216         if start >= end:
217             logger.error(f"{segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.")
218             return None, (idx, start, end, "start >= end")
219
220         # Check if segment extends beyond video duration
221         if video_duration > 0:
222             if start >= video_duration:
223                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is beyond the
video duration ({video_duration:.2f}s). "
224                             f"This segment cannot be extracted ? the video ran out.
Skipping.")
225                 return None, (idx, start, end, "start beyond video end")
226
227             if end > video_duration:
228                 original_end = end
229                 end = video_duration
230                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s) exceeds
video duration ({video_duration:.2f}s). "
231                             f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")
232
233             clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
234
235             # Precise sub-second trim using fast re-encoding to preserve stream headers
236             padded_start = max(0.0, start - self.begin_padding)
237             padded_end = end + self.end_padding
238
239             # Clamp padded_end to video duration if known
240             if video_duration > 0 and padded_end > video_duration:
241                 padded_end = video_duration
242                 logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s)
exceeds video duration. "
243                             f"Clamping to {video_duration:.2f}s (end padding partially
applied).")
244
245             trim_duration = padded_end - padded_start
246             logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
247
248             base_cmd = [
249                 "ffmpeg", "-y",
250                 "-ss", str(round(padded_start, 3)),
251                 "-to", str(round(padded_end, 3)),
252                 "-i", raw_video_path,
253                 "-c:v", "libx264",
254                 "-preset", "superfast",
255                 "-crf", "22",
256                 "-c:a", "aac",
257                 "-b:a", "128k",
258             ]
259             vf_args = ["-vf", watermark_filter] if watermark_filter else []
260
261             # Try with the watermark first; if that encode fails (e.g. a bad font/logo
262             # path), retry once WITHOUT it so a branding misconfig can never nuke the
263             # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
264             attempts = [vf_args, []] if vf_args else [[]]
265             produced = False
266             last_exc: Optional[subprocess.CalledProcessError] = None

```

```

267         cmd = base_cmd + [clip_path]
268     for attempt_vf in attempts:
269         cmd = base_cmd + attempt_vf + [clip_path]
270         try:
271             subprocess.run(cmd, capture_output=True, check=True)
272         except subprocess.CalledProcessError as e:
273             last_exc = e
274             if attempt_vf and vf_args:
275                 _err = (e.stderr.decode(errors="replace").strip()[-300:]
276                        if getattr(e, "stderr", None) else "")
277                 logger.warning(
278                     f"{segment_label}: watermark encode failed; retrying without it.
279 "
280                     f"ffmpeg: {_err or '(no stderr captured)'}"
281                 )
282                 continue
283             break
284         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
285             produced = True
286             if not attempt_vf and vf_args:
287                 logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the
288 watermark filter failed).")
289             break
290         last_exc = None # ffmpeg succeeded but produced nothing
291         break
292     if produced:
293         return clip_path, None
294     elif last_exc is not None:
295         stderr_msg = last_exc.stderr.decode(errors='replace').strip()
296         logger.error(f"{segment_label}: FFmpeg trim failed.")
297         print(f" Command: {' '.join(cmd)}")
298         print(f" FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to avoid
299 flooding
300         return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
301     else:
302         logger.error(f"{segment_label}: FFmpeg exited successfully but output clip
303 is missing or empty. Skipping.")
304         return None, (idx, start, end, "empty output clip")
305
306     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
307 float]], output_filename: str) -> str:
308         """
309         Extracts selected segments from a raw video and stitches them together
310         into a seamless highlights file, applying end_padding to prevent abrupt endings.
311         """
312         if not segments:
313             raise ValueError("No highlight segments provided to compile.")
314
315         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
316         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
317         original_count = len(segments)
318         segments = self._merge_overlapping(segments)
319         if len(segments) < original_count:
320             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
321 -> %d.",
322                         original_count - len(segments), original_count, len(segments))
323         if not segments:
324             raise ValueError("No valid (start < end) highlight segments after merge.")
325
326         if not os.path.exists(raw_video_path):
327             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
328 {raw_video_path}")
329
330         # Defense-in-depth: never let the output name escape output_dir (the API also

```

```

325         # validates this at the request boundary).
326         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
327
328         # Probe video duration so we can detect out-of-range segments
329         video_duration = self._get_video_duration(raw_video_path)
330         if video_duration > 0:
331             logger.info(f"Source video duration: {video_duration:.2f}s")
332         else:
333             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
334
335         # We will create temporary clips and stitch them using FFmpeg concat demuxer
336         temp_clips = []
337         skipped_segments = []
338
339         # Create a temp directory within output_dir
340         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
341             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
342             # Build the (optional) watermark filtergraph once; it is applied during
every
343             # per-clip re-encode so the mark is baked into each clip identically.
344             watermark_filter = self._watermark_filter(tmp_dir)
345             if watermark_filter:
346                 logger.info("[watermark] applying branding overlay to each clip.")
347             for idx, (raw_start, raw_end) in enumerate(segments):
348                 clip_path, skip_record = self._trim_one_segment(
349                     idx, raw_start, raw_end, segments, video_duration,
350                     tmp_dir, raw_video_path, watermark_filter,
351                 )
352                 if clip_path is not None:
353                     temp_clips.append(clip_path)
354                 if skip_record is not None:
355                     skipped_segments.append(skip_record)
356
357             # Report summary of skipped segments
358             if skipped_segments:
359                 logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
360                 for seg_idx, seg_start, seg_end, reason in skipped_segments:
361                     logger.info(f"    - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
362                 logger.info("")
363
364             if not temp_clips:
365                 raise RuntimeError(
366                     f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
367                     f"All {len(segments)} segments were skipped. "
368                     f"Check the errors above for details. "
369                     f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
370                 )
371
372             logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
373
374             # Create a list file for FFmpeg concat demuxer
375             list_file_path = os.path.join(tmp_dir, "clips.txt")
376             # encoding pinned: a non-ASCII clip path would crash on Windows (cp1252
default);
377             # ffmpeg's concat demuxer reads the list as UTF-8 on every OS.
378             with open(list_file_path, "w", encoding="utf-8") as f:
379                 for clip in temp_clips:
380                     # Escape single quotes in path
381                     safe_path = clip.replace("'", "'\\'")

```

```

382         f.write(f"file '{safe_path}'\n")
383
384     logger.info("Stitching clips together...")
385     # Stitch them together without re-encoding (since they were encoded with
identical settings)
386     concat_cmd = [
387         "ffmpeg", "-y",
388         "-f", "concat",
389         "-safe", "0",
390         "-i", list_file_path,
391         "-c", "copy",
392         output_path
393     ]
394
395     try:
396         subprocess.run(concat_cmd, capture_output=True, check=True)
397
398         # Final verification
399         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
400             raise RuntimeError(
401                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
402                 f"but output file is missing or empty: {output_path}"
403             )
404
405         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
406         logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
407         return output_path
408     except subprocess.CalledProcessError as e:
409         stderr_msg = e.stderr.decode(errors='replace').strip()
410         raise RuntimeError(
411             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
412             f"Command: {' '.join(concat_cmd)}\n"
413             f"FFmpeg stderr: {stderr_msg[-500:]}"
414         ) from e
415

```

7. user (2026-06-21T02:34:31.450Z)

EISDIR: illegal operation on a directory, read 'C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors'

8. assistant (2026-06-21T02:34:33.501Z)

Let me read the detector files individually:

9. user (2026-06-21T02:34:33.764Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6
7     The ABC lets us:
8     - Keep the current WSL adapters working unchanged for Windows users.
9     - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10    the hybrid segmenters or the model-agnostic windowing brain.
11    - Provide a single place to evolve the healthcheck / error contract and thread
12    detector readiness into per-video reports.
13
14    Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15    - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16    Run detector, return the Windows-side path to the produced trajectory CSV

```

```

17     (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18     The caller (hybrid) is responsible for output dir; impls may stage the video
19     into WSL fs for perf.
20
21     - healthcheck() -> tuple[bool, str]
22     Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23     Message is logged (and can be surfaced to reports or UX in future).
24     Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26 Existing behavior is 100% preserved. The only additions are the inheritance and
27 the explicit common type for future implementers.
28
29 How to add a new runner (for docs / future contributors):
30 1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31 2. Implement run_predict and healthcheck (return (bool, str) tuple).
32 3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33    TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34    deliberately static and model-agnostic). If the new detector needs different
35    windowing, it can implement its own or we generalize later.
36 4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37    registry never pulls torch/tf on a machine that doesn't have them).
38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39    DetectorRunner instances (they already do the right thing via duck-typing + the
40    type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42    returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44    windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 """
68
69 import logging
70 import shlex
71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash

```



```

82     treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default ``~/clips``
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/\" + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100    and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101    ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102    """
103
104    cfg: Any
105
106    def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107        full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108        argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109        logger.debug("WSL exec: %s", full)
110        return subprocess.run(argv, capture_output=True, text=True,
111                              timeout=(self.cfg.timeout_sec or None))
112
113    # ---- cache hygiene helpers (shared by both WSL runners) -----
114    def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115        """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117        Returns None if it can't be determined (never raises). Used as a pre-run
118        disk guard so a long extraction doesn't fill the disk mid-flight.
119        """
120        try:
121            res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122            'NR==2{{print $4}}'")
123            except (subprocess.TimeoutExpired, OSError):
124                return None
125            try:
126                return int((res.stdout or "").strip()) / (1024 ** 3)
127            except (ValueError, AttributeError):
128                return None
129
130    def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131        """Best-effort recursive delete of WSL paths (post-success cleanup; never
132        raises)."""
133        paths = [p for p in wsl_paths if p]
134        if not paths:
135            return
136        quoted = " ".join(wsl_tilde_quote(p) for p in paths)
137        try:
138            self._wsl_bash(f"rm -rf {quoted}")
139        except (subprocess.TimeoutExpired, OSError) as e:
140            logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
141
142    def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
143        """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
144
145        Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
146        filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path

```

```

145         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
146         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
147         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
148         any
149         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
150         ``cp`` still gets its chance (we never block a genuinely-readable source).
151         """
152         try:
153             res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
154         except (subprocess.TimeoutExpired, OSError):
155             return None
156         if "READABLE" in (res.stdout or ""):
157             return None
158         return (
159             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
160             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
161             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
162         )
163
164     class DetectorRunner(ABC):
165         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167         @abstractmethod
168         def run_predict(self, video_win_path: str, output_win_dir: str,
169                       video_id: Optional[str] = None) -> Optional[str]:
170             """Run inference and return path to trajectory CSV (or None on failure).
171
172             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173             videos that happen to share a filename cannot reuse each other's frames or CSV.
174             It is optional for back-compat; callers in the live pipeline always pass it.
175             """
176             pass
177
178         @abstractmethod
179         def healthcheck(self) -> Tuple[bool, str]:
180             """Quick check if the runner (env, weights, etc.) is usable.
181
182             Returns (ok, message). Never raises. Cheap to call before a long run.
183             """
184             pass
185
186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192

```

10. user (2026-06-21T02:34:34.040Z)

```

1     """CPU stub shuttle-trajectory runner ? "mock a GPU with a CPU".
2
3     A ``DetectorRunner`` that produces a deterministic ``Frame,Visibility,X,Y`` trajectory
4     CSV with no GPU, no WSL, no model weights ? so the whole trajectory-hybrid pipeline
5     (``wasb_hybrid`` / ``tracknet_hybrid``) can run end-to-end on a CPU-only box (CI, a CPU
6     container, a laptop) for regression testing. The person/yolo path already CPU-falls-back
7     (``use_gpu=False``); this closes the last GPU/WSL-only gap so the entire pipeline is
8     runnable ? and regression-gateable ? without a GPU.
9
10    Two modes:
11    * replay ? emit an existing ``Frame,Visibility,X,Y`` CSV verbatim (drive the
12      pipeline with a committed/golden trajectory; deterministic + cross-OS).
13    * synth (default) ? generate a deterministic synthetic trajectory of

```

```

14     ``num_rallies`` in-flight bursts (visible + fast-moving frames) separated by dead
15     gaps, so windowing yields a stable, known number of candidate windows.
16
17     This is the CPU seam the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
18     "Optional Linux-native path") and the **seed of the future ``LinuxNativeRunner``**
19     (M3.5, ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``). It is selected via
20     ``indexing.detector_impl="stub"`` or the ``RALLY_DETECTOR_IMPL=stub`` env var; the real
21     WSL runners stay the default, so production behaviour is unchanged.
22     """
23     import os
24     import csv as _csv
25     import shutil
26     import logging
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional, Tuple
29
30     from backend.pipeline.detectors.base import DetectorRunner
31     # Reuse the model-agnostic CSV parsing + windowing (no torch/tf pulled here).
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33
34     logger = logging.getLogger(__name__)
35
36
37     @dataclass
38     class StubConfig:
39         """Settings for the CPU stub runner (built from ``config.json`` ->
40         ``indexing.stub``)."""
41         fps: float = 30.0
42         frame_width: float = 1280.0
43         num_rallies: int = 2
44         rally_seconds: float = 4.0
45         gap_seconds: float = 3.0          # dead gap between rallies (> merge_gap so
46         windows don't merge)
47         lead_seconds: float = 2.0        # dead lead-in
48         step_px: float = 12.0            # per-frame shuttle displacement during a rally
49         (>> velocity_thresh)
50         replay_csv: Optional[str] = None # if set + exists, emit this CSV verbatim instead
51         of synthesising
52
53     @classmethod
54     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "StubConfig":
55         s = (idx_cfg or {}).get("stub", {}) or {}
56         return cls(
57             fps=float(s.get("fps", 30.0)),
58             frame_width=float(s.get("frame_width", 1280.0)),
59             num_rallies=int(s.get("num_rallies", 2)),
60             rally_seconds=float(s.get("rally_seconds", 4.0)),
61             gap_seconds=float(s.get("gap_seconds", 3.0)),
62             lead_seconds=float(s.get("lead_seconds", 2.0)),
63             step_px=float(s.get("step_px", 12.0)),
64             replay_csv=s.get("replay_csv"),
65         )
66
67     class StubDetectorRunner(DetectorRunner):
68         """Deterministic CPU trajectory runner (no GPU/WSL). See the module docstring."""
69
70         def __init__(self, cfg: Optional[StubConfig] = None):
71             self.cfg = cfg or StubConfig()
72
73         def healthcheck(self) -> Tuple[bool, str]:
74             return True, "stub CPU detector (no GPU/WSL)"
75
76         def _rows(self) -> List[Tuple[int, int, float, float]]:
77             """Deterministic ``(frame, visible, x, y)`` rows: ``num_rallies`` in-flight

```

```

bursts
75         separated by dead gaps. During a rally the shuttle oscillates ``step_px``/frame
?
76         well above the default 0.02 velocity threshold ? so every rally frame is
"active"
77         and each burst becomes exactly one candidate window.""
78         cfg = self.cfg
79         rows: List[Tuple[int, int, float, float]] = []
80         frame = 0
81         y = cfg.frame_width / 4.0
82         base_x = cfg.frame_width / 2.0
83         for _ in range(int(cfg.lead_seconds * cfg.fps)):
84             rows.append((frame, 0, 0.0, 0.0))
85             frame += 1
86         for r in range(cfg.num_rallies):
87             for i in range(int(cfg.rally_seconds * cfg.fps)):
88                 x = base_x + (cfg.step_px if i % 2 else -cfg.step_px)
89                 rows.append((frame, 1, x, y))
90                 frame += 1
91             if r != cfg.num_rallies - 1:
92                 for _ in range(int(cfg.gap_seconds * cfg.fps)):
93                     rows.append((frame, 0, 0.0, 0.0))
94                     frame += 1
95         return rows
96
97     def run_predict(self, video_win_path: str, output_win_dir: str,
98                    video_id: Optional[str] = None) -> Optional[str]:
99         """Write a deterministic trajectory CSV and return its path (never touches the
GPU/WSL)."""
100         os.makedirs(output_win_dir, exist_ok=True)
101         stem = os.path.splitext(os.path.basename(str(video_win_path).replace("\\",
"/")))[0]
102         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "stub")
103         out_csv = os.path.join(output_win_dir, f"{key}_stub.csv")
104
105         if self.cfg.replay_csv and os.path.exists(self.cfg.replay_csv):
106             shutil.copyfile(self.cfg.replay_csv, out_csv)
107             logger.info("Stub runner: replayed trajectory %s -> %s",
self.cfg.replay_csv, out_csv)
108         return out_csv
109
110         rows = self._rows()
111         with open(out_csv, "w", newline="") as f:
112             w = _csv.writer(f)
113             w.writerow(["Frame", "Visibility", "X", "Y"])
114             for frame, vis, x, y in rows:
115                 w.writerow([frame, vis, f"{x:.3f}", f"{y:.3f}"])
116         logger.info("Stub runner: wrote %d synthetic trajectory rows -> %s", len(rows),
out_csv)
117         return out_csv
118
119         # Model-agnostic CSV parsing + windowing ? identical to the real runners.
120         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
121         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
122
123
124     def synth_trajectory_from_labels(labels_csv: str, out_csv: str, *, fps: float = 30.0,
125                                    frame_width: float = 1280.0, tail_pad: float = 2.0,
126                                    step_px: float = 12.0) -> str:
127         """CPU "mock GPU" for TRAINING: write a deterministic Frame,Visibility,X,Y
trajectory whose
128         in-flight (visible + oscillating) frames cover each labeled rally window and whose
dead frames
129         cover the gaps ? so the synthetic trajectory *encodes* the rally labels.

```

```

130
131     This lets the Gen-0 harness train + leave-one-video-out eval end-to-end on coherent
data with
132     no GPU/WSL, validating the whole training pipeline before the real WASB detector
lands. (The
133     learned model is meaningful for plumbing, not for production ? the trajectory is
synthetic.)
134     """
135     rallies: List[Tuple[float, float]] = []
136     with open(labels_csv, newline="", encoding="utf-8") as f:
137         for row in _csv.DictReader(f):
138             try:
139                 rallies.append((float(row["start_time"]), float(row["end_time"])))
140             except (KeyError, ValueError):
141                 continue
142     duration = (max(e for _, e in rallies) + tail_pad) if rallies else 10.0
143     nframes = max(1, int(duration * fps))
144     base_x = frame_width / 2.0
145     y = frame_width / 4.0
146     os.makedirs(os.path.dirname(out_csv) or ".", exist_ok=True)
147     with open(out_csv, "w", newline="", encoding="utf-8") as f:
148         w = _csv.writer(f)
149         w.writerow(["Frame", "Visibility", "X", "Y"])
150         for i in range(nframes):
151             t = i / fps
152             if any(s <= t < e for s, e in rallies):
153                 x = base_x + (step_px if i % 2 else -step_px)
154                 w.writerow([i, 1, f"{x:.3f}", f"{y:.3f}"])
155             else:
156                 w.writerow([i, 0, "0.000", "0.000"])
157     return out_csv
158

```

11. user (2026-06-21T02:34:34.196Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24    logger = logging.getLogger(__name__)
25
26    # Windows path to the inference wrapper we stage into WSL.
27    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28

```

```

29
30 @dataclass
31 class WasbConfig:
32     """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/WASB-SBDT"
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "wasb"
37     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38     sport: str = "badminton"
39     wsl_stage_dir: str = "~/clips"
40     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41     keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42     min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45     # extraction that dominates a long clip. Output is bit-identical to the PNG path
46     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47     stream_video: bool = False
48
49 @classmethod
50 def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51     w = (idx_cfg or {}).get("wasb", {}) or {}
52     return cls(
53         distro=w.get("wsl_distro", "Ubuntu"),
54         repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55         conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56         conda_env=w.get("conda_env", "wasb"),
57         weights_path=w.get("weights_path",
58             "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59         sport=w.get("sport", "badminton"),
60         wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61         timeout_sec=int(w.get("timeout_sec", 1800)),
62         keep_frames=bool(w.get("keep_frames", False)),
63         min_free_gb=float(w.get("min_free_gb", 0.0)),
64         stream_video=bool(w.get("stream_video", False)),
65     )
66
67
68 class WasbRunner(WslCommandMixin, DetectorRunner):
69     def __init__(self, cfg: WasbConfig):
70         self.cfg = cfg
71
72     def healthcheck(self) -> Tuple[bool, str]:
73         """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74         cmd = (f"conda activate {self.cfg.conda_env} && "
75             f"python -c \"import torch; print('torch', torch.__version__, "
76             f"'cuda', torch.cuda.is_available())\"")
77         try:
78             res = self._wsl_bash(cmd)
79         except FileNotFoundError:
80             return False, "`wsl` not found ? Windows + WSL2 required."
81         except subprocess.TimeoutExpired:
82             return False, "WSL healthcheck timed out."
83         out = (res.stdout or "").strip()
84         if res.returncode != 0:
85             return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86         return True, out
87

```

```

88     def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89         src_mnt = to_wsl_mnt_path(video_win_path)
90         reason = self.wsl_source_unreadable_reason(src_mnt)
91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96         try:
97             res = self._wsl_bash(cmd)
98         except subprocess.TimeoutExpired:
99             logger.error("Staging video into WSL timed out.")
100            return None
101            if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                return None
104            return dest
105
106    def _stage_infer_script(self) -> bool:
107        """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108        src_mnt = to_wsl_mnt_path(_INFER_WIN)
109        cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110        res = self._wsl_bash(cmd)
111        return res.returncode == 0 and "OK" in (res.stdout or "")
112
113    def run_predict(self, video_win_path: str, output_win_dir: str,
114                    video_id: Optional[str] = None) -> Optional[str]:
115        """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117        All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118        two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119        can never reuse each other's staged frames, resumable cache, or CSV.
120
121        # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122        # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123        # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124        # the CSV landed inside WSL while Windows polled the repo-relative path.
125        output_win_dir = os.path.abspath(output_win_dir)
126        os.makedirs(output_win_dir, exist_ok=True)
127        # Normalize for cross-platform basename (tests use Windows paths on Linux).
128        video_win_path = str(video_win_path).replace("\\", "/")
129        base = os.path.basename(video_win_path)
130        stem, ext = os.path.splitext(base)
131        # Unique, content-derived key: same filename + different bytes -> different key.
132        key = f"{stem}__{video_id[:12]}" if video_id else stem
133        wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134        expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136        if not self._stage_infer_script():
137            logger.error("Could not stage wasb_infer.py into WSL.")
138            return None
139        staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140        if staged_video is None:
141            return None
142        frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144        # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145        # WSL stage filesystem is already low so the user can clean up before it fills.
146        # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147        if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148            free = self._wsl_free_gb(self.cfg.wsl_stage_dir)

```

```

149         if free is not None and free < self.cfg.min_free_gb:
150             logger.warning(
151                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                 "may fill the disk ? consider running tools/cleanup_caches.py.",
153                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "
159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161         cmd = (
162             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163             f"python wasb_infer.py "
164             f"--video {wsl_tilde_quote(staged_video)} "
165             f"{frame_args}"
166             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167             f"--sport {shlex.quote(self.cfg.sport)} "
168             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
169         )
170         logger.info("Running WASB inference on %s ...", base)
171         try:
172             res = self._wsl_bash(cmd)
173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

12. user (2026-06-21T02:34:34.510Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2

```



```

3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10    * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11    ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13    * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
14    into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
15    * weights / repo / python / scratch come from the environment first
16    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
17    ``indexing.wasb`` config, then a default ? so a box needs no config edits.
18
19    It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__vid12}.wasb.csv``
20    name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
21    ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
22    the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
23    (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
24
25    Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
26    (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
27    candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
28    ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
29    run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
30    vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
31    artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
32    (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
33    not the numbers.
34    """
35    import os
36    import shutil
37    import logging
38    import subprocess
39    from dataclasses import dataclass
40    from typing import Any, Dict, List, Optional, Tuple
41
42    from backend.pipeline.detectors.base import DetectorRunner
43    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
44    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
45
46    logger = logging.getLogger(__name__)
47
48    # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
49    # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
50    _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
51
52
53    @dataclass
54    class NativeWasbConfig:
55        """Resolved settings for a Linux-native WASB run.

```

```

56
57     Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
58     over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
59     ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
60     perf win).
61     """
62     repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
63     python_bin: str = "python"           # the `wasb` conda env python (invoked by
64     path, no activate)
65     weights_path: str = ""                # REQUIRED (env WASB_WEIGHTS or
66     indexing.wasb.weights_path)
67     sport: str = "badminton"
68     device: str = "cuda"                  # cuda | mps | cpu
69     scratch_dir: str = ""                 # frame-extraction scratch (env); "" = next
70     to the output dir
71     timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no
72     timeout)
73     keep_frames: bool = False             # retain the decoded frame cache after
74     success (debug only)
75     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
76     clip's
77     # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
78     real GPU
79     # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
80     here.
81     stream_video: bool = True
82
83     @classmethod
84     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
85         w = (idx_cfg or {}).get("wasb", {}) or {}
86         env = os.environ.get
87         # stream_video is tri-state in config: None/absent => native default (stream);
88         an
89         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
90         sv = w.get("stream_video")
91         return cls(
92             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
93             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
94             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
95             sport=w.get("sport", "badminton"),
96             device=env("WASB_DEVICE") or w.get("device", "cuda"),
97             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
98             "",
99             timeout_sec=int(w.get("timeout_sec", 1800)),
100             keep_frames=bool(w.get("keep_frames", False)),
101             stream_video=(True if sv is None else bool(sv)),
102         )
103
104     class NativeWasbRunner(DetectorRunner):
105         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
106         docstring."""
107
108         def __init__(self, cfg: NativeWasbConfig):
109             self.cfg = cfg
110
111         # --- low-level native exec (the single place tests patch)
112         -----
113         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
114         subprocess.CompletedProcess:
115             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
116             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
117                                   timeout=(self.cfg.timeout_sec or None))

```

```

106     def _repo_src(self) -> str:
107         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
108
109     def _copy_infer_script(self) -> bool:
110         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
111         configs/."""
112         repo_src = self._repo_src()
113         try:
114             os.makedirs(repo_src, exist_ok=True)
115             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
116             return True
117         except OSError as e:
118             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
119             return False
120
121     def _rm_rf(self, path: Optional[str]) -> None:
122         """Best-effort recursive delete of a native path (post-success cleanup; never
123         raises)."""
124         if path:
125             shutil.rmtree(path, ignore_errors=True)
126
127     def healthcheck(self) -> Tuple[bool, str]:
128         """Verify weights + repo present and the `wasb` python imports torch (and sees a
129         GPU on cuda)."""
130         if not self.cfg.weights_path:
131             return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
132             indexing.wasb.weights_path).")
133         weights = os.path.expanduser(self.cfg.weights_path)
134         if not os.path.exists(weights):
135             return False, f"WASB weights not found at {weights}"
136         repo_src = self._repo_src()
137         if not os.path.isdir(repo_src):
138             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
139             nttcom/WASB-SBDT "
140             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
141         code = "import torch; print('torch', torch.__version__, 'cuda',
142         torch.cuda.is_available())"
143         try:
144             res = self._run([self.cfg.python_bin, "-c", code])
145         except FileNotFoundError:
146             return False, f"python binary not found: {self.cfg.python_bin!r} (set
147             WASB_PYTHON)."
148         except subprocess.TimeoutExpired:
149             return False, "native torch healthcheck timed out."
150         out = (res.stdout or "").strip()
151         if res.returncode != 0:
152             return False, f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
153             out).strip()}"
154         if self.cfg.device == "cuda" and "cuda True" not in out:
155             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
156         return True, out
157
158     def run_predict(self, video_win_path: str, output_win_dir: str,
159                     video_id: Optional[str] = None) -> Optional[str]:
160         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
161         None.
162
163         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
164         WSL
165         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
166         the
167         same downstream contract, so the parity comparison is apples-to-apples.
168         """
169         output_dir = os.path.abspath(output_win_dir)
170         os.makedirs(output_dir, exist_ok=True)

```

```

160         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
161         norm = str(video_win_path).replace("\\", "/")
162         stem, _ext = os.path.splitext(os.path.basename(norm))
163         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
164         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
165         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
166
167         if not self._copy_infer_script():
168             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
169             return None
170
171         weights = os.path.expanduser(self.cfg.weights_path)
172         argv = [self.cfg.python_bin, "wasb_infer.py",
173               "--video", os.path.abspath(str(video_win_path))]
174         frames_dir: Optional[str] = None
175         if self.cfg.stream_video:
176             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
177             argv.append("--stream-video")
178         else:
179             scratch = self.cfg.scratch_dir or output_dir
180             frames_dir = os.path.join(scratch, f"{key}_frames")
181             argv += ["--frames_out_dir", frames_dir]
182             argv += ["--weights", weights,
183                   "--sport", self.cfg.sport,
184                   "--device", self.cfg.device,
185                   "--out", out_csv]
186
187         logger.info("Running native WASB inference (device=%s) on %s ...",
188                   self.cfg.device, os.path.basename(norm))
189         try:
190             res = self._run(argv, cwd=self._repo_src())
191         except FileNotFoundError:
192             logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
193             return None
194         except subprocess.TimeoutExpired:
195             logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
196             return None
197         if res.returncode != 0:
198             logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
199             return None
200         if not os.path.exists(out_csv):
201             logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
202             return None
203         logger.info("WASB trajectory CSV: %s", out_csv)
204
205         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
206         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
207         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
208         if not self.cfg.keep_frames and frames_dir:
209             logger.info("Cache hygiene: removing frame cache for %s "
210                       "(set indexing.wasb.keep_frames=true to retain).", key)
211             self._rm_rf(frames_dir)
212         return out_csv
213
214         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).

```

```

215         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
216         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
217

```

13. assistant (2026-06-21T02:34:37.218Z)

Now let me check the tracknet_runner for the shared contract:

14. user (2026-06-21T02:34:37.979Z)

```

1      """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4      inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6          1. Translate the Windows video path to its WSL-visible form (and optionally
7             stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8          2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9          3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10         4. Convert that trajectory into high-motion "action windows" ? the same
11            candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12            pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5
39         stage_in_wsl: bool = True                     # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                       # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                     # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":

```

```

47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WslCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.
92         """
93
94         def __init__(self, cfg: TrackNetConfig):
95             self.cfg = cfg
96
97         def healthcheck(self) -> Tuple[bool, str]:
98             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100             Returns (ok, message). Cheap to call before a long run.
101             """
102             cmd = (
103                 f"conda activate {self.cfg.conda_env} && "
104                 f"python -c \"import tensorflow as tf; "
105                 f"print('TF', tf.__version__, 'GPUs', "
106                 f"len(tf.config.list_physical_devices('GPU')))\n"
107             )
108             try:
109                 res = self._wsl_bash(cmd)
110             except FileNotFoundError:
111                 return False, "wsl` not found ? is this running on Windows with WSL2

```

```

installed?"
111         except subprocess.TimeoutExpired:
112             return False, "WSL healthcheck timed out."
113         out = (res.stdout or "").strip()
114         if res.returncode != 0:
115             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
116         return True, out
117
118     # ----- inference
119
120     def run_predict(self, video_win_path: str, output_win_dir: str,
121                    video_id: Optional[str] = None) -> Optional[str]:
122         """Run predict.py on a video. Returns the Windows path to the produced CSV, or
123         None.
124
125         ``output_win_dir`` is a Windows directory (under the repo's output/) that
126         WSL writes into via its /mnt view, so the Windows process can read the CSV
127         back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
128         therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
129         a filename cannot collide on the output CSV.
130         """
131         if not self.cfg.weights_path:
132             logger.error(
133                 "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
134                 "in config.json to the WSL path of your trained TrackNetV4 weights."
135             )
136             return None
137
138         # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
139         # relative paths through unchanged, so WSL would resolve them against the
140         # predict.py cwd instead of the repo (same bug class as wasb_runner).
141         output_win_dir = os.path.abspath(output_win_dir)
142         os.makedirs(output_win_dir, exist_ok=True)
143         # Normalize separators so basename/splitext work when tests (or collector)
144         # pass Windows-style paths on a Linux host.
145         video_win_path = str(video_win_path).replace("\\", "/")
146         base = os.path.basename(video_win_path)
147         stem, ext = os.path.splitext(base)
148
149         # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
150         if ext.lower() not in (".mp4", ".avi"):
151             logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
152             return None
153
154         wsl_out = to_wsl_mnt_path(output_win_dir)
155         # Namespace by video_id so two same-filename videos don't collide on the CSV.
156         # predict.py derives its output name from the (staged) video stem, so staging
157         # under the namespaced name propagates into "<key>_predict.csv".
158         key = f"{stem}_{video_id[:12]}" if video_id else stem
159         expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
160
161         # Resolve the video path WSL will read.
162         if self.cfg.stage_in_wsl:
163             staged = self._stage_video(video_win_path, f"{key}{ext}")
164             if staged is None:
165                 return None
166             wsl_video = staged
167         else:
168             # No staging: predict.py reads /mnt directly and writes
169             # "<stem>_predict.csv".
170             # We cannot rename its output, so fall back to the un-namespaced name.
171             wsl_video = to_wsl_mnt_path(video_win_path)
172             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
173
174         if self.cfg.min_free_gb > 0:

```

```

173         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174         if free is not None and free < self.cfg.min_free_gb:
175             logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                             "consider running tools/cleanup_caches.py.",
177                             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}")
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
193                         self.cfg.timeout_sec)
194             return None
195
196         if res.returncode != 0:
197             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198                 res.stdout)[-2000:])
199             return None
200
201         if not os.path.exists(expected_csv_win):
202             logger.error("TrackNet finished but expected CSV not found at %s",
203                         expected_csv_win)
204             return None
205         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
206
207         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
208         # regenerable intermediate). On earlier failure we return above without
209         cleaning.
210         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
211             logger.info("Cache hygiene: removing staged video for %s "
212                         "(set indexing.tracknet.keep_frames=true to retain).", key)
213             self._wsl_rm_rf([wsl_video])
214             return expected_csv_win
215
216     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
217         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
218
219         Returns the WSL path to the staged copy, or None on failure.
220         """
221         src_mnt = to_wsl_mnt_path(video_win_path)
222         reason = self.wsl_source_unreadable_reason(src_mnt)
223         if reason:
224             logger.error("Cannot stage video into WSL: %s", reason)
225             return None
226         dest = f"{self.cfg.wsl_stage_dir}/{base}"
227         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
228         {wsl_tilde_quote(dest)} && echo OK"
229         try:
230             res = self._wsl_bash(cmd)
231         except subprocess.TimeoutExpired:
232             logger.error("Staging video into WSL timed out.")
233             return None
234         if res.returncode != 0 or "OK" not in (res.stdout or ""):
235             logger.error("Failed to stage video into WSL: %s", (res.stderr or
236                 res.stdout).strip())
237             return None

```



```

232         return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252         points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def _active_frames(points: List[TrajectoryPoint], fps: float, frame_width: float,
257                        velocity_thresh: float) -> List[Tuple[int, float]]:
258         """Per-frame ``(frame_idx, velocity)`` for frames where the shuttle is in
flight.
259
260         Computes each visible point's normalised velocity from the last *visible* point;
261         an absent shuttle breaks the trajectory. A frame qualifies when its velocity
exceeds
262         ``velocity_thresh``. ``fps``/``frame_width`` are expected to be already
defaulted by
263         the caller so identical floats feed the velocity math."""
264         active = [] # (frame_idx, velocity)
265         prev: Optional[TrajectoryPoint] = None
266         for p in points:
267             if not p.visible:
268                 prev = None # break the trajectory; absent shuttle = not in flight
269                 continue
270             if prev is not None:
271                 dframes = p.frame - prev.frame
272                 if dframes > 0:
273                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
274                     velocity = (dist / frame_width) * (fps / dframes)
275                     if velocity > velocity_thresh:
276                         active.append((p.frame, velocity))
277             prev = p
278         return active
279
280     @staticmethod
281     def trajectory_to_action_windows(
282         points: List[TrajectoryPoint],
283         fps: float,
284         frame_width: float,
285         chunk_start: float = 0.0,
286         velocity_thresh: float = 0.02,
287         merge_gap: float = 2.0,
288         min_window_duration: float = 2.0,
289         chunk_index: Optional[int] = None,
290         max_window_duration: float = 0.0,
291         min_split_gap: float = 0.5,
292         window_hard_cap: bool = False,
293         window_inactivity_end: float = 0.0,

```

```

294         inactivity_min_density: float = 0.34,
295         inactivity_rally_thresh: float = 0.08,
296         window_blob_fallback: bool = False,
297         window_blob_factor: float = 4.0,
298     ) -> List[Dict[str, Any]]:
299         """Convert a shuttle trajectory into candidate rally windows.
300

```

15. user (2026-06-21T02:34:38.293Z)

```

87: class TrackNetRunner(WslCommandMixin, DetectorRunner):
237:     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
281:     def trajectory_to_action_windows(

```

16. assistant (2026-06-21T02:34:40.447Z)

Let me also check for filesystem coupling points and look at the API/endpoints:

17. user (2026-06-21T02:34:41.327Z)

```

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/eval/gemini_refine.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/gemini.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/trajectory_hybrid.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/yolo_hybrid.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/utils/video.py

```

18. user (2026-06-21T02:34:43.081Z)

```

1     import subprocess
2     import logging
3     from typing import Optional
4
5     logger = logging.getLogger(__name__)
6
7     def get_video_duration(video_path: str) -> float:
8         """Returns the exact duration of a video file in seconds using ffprobe.
9         Returns 0.0 on failure."""
10        try:
11            duration_output = subprocess.check_output([
12                "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                "-of", "default=noprint_wrappers=1:nokey=1", video_path
14            ]).decode().strip()
15            return float(duration_output)
16        except Exception as e:
17            logger.error(f"Could not detect video duration for {video_path}: {e}")
18            return 0.0
19
20    def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                           output_path: str, reencode: bool = False,
22                           scale_width: Optional[int] = None) -> bool:
23        """Extracts a chunk from a video file.
24
25        Args:
26            reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                     If False, uses stream-copy (fast but cuts at nearest keyframe).
28                     Use True for short clips where keyframe drift matters.
29            scale_width: If set (requires reencode), downscale to this width (height auto,
30                       even). Useful for high-res sources headed to an API that analyses at
31                       low res anyway ? shrinks uploads with no analysis loss.
32
33        """
34        cmd = [

```

```

34         "ffmpeg", "-y",
35         "-ss", str(start_time),
36         "-t", str(duration),
37         "-i", video_path,
38     ]
39     if reencode:
40         if scale_width:
41             cmd += ["-vf", f"scale={int(scale_width)}:-2"]
42             cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
43         else:
44             cmd += ["-c", "copy"]
45         cmd.append(output_path)
46         try:
47             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
48             return True
49         except Exception as e:
50             logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s:
{e}")
51         return False
52

```

19. user (2026-06-21T02:34:43.896Z)

```

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/api/job_worker.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/api/uploads.py
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/api/__init__.py

```

20. user (2026-06-21T02:34:46.161Z)

```

1     """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...) ` still intercepts the inter-function calls (e.g. a parked job arms the `patched`
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18    import logging
19    import threading
20    import traceback
21    from concurrent.futures import ThreadPoolExecutor
22    from typing import Any, Dict, Optional
23
24    logger = logging.getLogger(__name__)
25
26    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
27    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
28    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
29    # already parallelize their AI handoff internally via their own pools.
30    _job_executor: Optional[ThreadPoolExecutor] = None
31
32

```

```

33     def _get_executor() -> ThreadPoolExecutor:
34         """Lazy module-global executor; tests monkeypatch this for inline execution."""
35         global _job_executor
36         if _job_executor is None:
37             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
38         return _job_executor
39
40
41     class JobWaiting(Exception):
42         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
43         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
44         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
45         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
46         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
47
48         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
49         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
50         """
51
52
53         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
54             super().__init__(f"job parked for {delay_sec:.0f}s")
55             self.delay_sec = max(0.0, float(delay_sec))
56             self.progress = progress
57
58
59     def _job_to_request(job: Dict[str, Any]):
60         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
61         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
62         the original submit. The row stores exactly the ProcessRequest fields."""
63         import backend.main as _main
64         return _main.ProcessRequest(
65             video_path=job["video_path"],
66             player_pool=job.get("player_pool"),
67             max_frames=job.get("max_frames"),
68             segmenter=job.get("segmenter"),
69         )
70
71
72     def _run_claimed_job(job: Dict[str, Any]) -> None:
73         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79         job self-scheduled and will be re-claimed when `next_poll_at` is due.
80         """
81         import backend.main as _main
82         db_instance = _main.db_instance
83         job_id = job["id"]
84         req = _main._job_to_request(job)
85         try:
86             result = _main._execute_processing(
87                 req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88             if result.get("video_id"):
89                 db_instance.set_job_video_id(job_id, result["video_id"])
90             db_instance.mark_job_done(job_id, result)

```

```

91     except _main.JobWaiting as w:
92         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
93         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95     except Exception as e:
96         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100 def _drain_due_jobs() -> int:
101     """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105     up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106     a single-worker box keeps making progress with no central timer (the DB is the
clock).
107     Returns the number of jobs that reached a terminal/parked state this tick.
108     """
109     import backend.main as _main
110     ran = 0
111     while True:
112         job = _main.db_instance.claim_due_job()
113         if job is None:
114             break
115         ran += 1
116         _main._run_claimed_job(job)
117     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
118     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
119     # on the next submit/drain. Best-effort ? never raises into the worker.
120     _main._schedule_next_drain_if_waiting()
121     return ran
122
123
124 def _schedule_next_drain_if_waiting() -> None:
125     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126     self-scheduled job resumes with no further client submit. The drain itself re-checks
127     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128     import backend.main as _main
129     try:
130         delay = _main.db_instance.seconds_until_next_due()
131     except Exception as e: # pragma: no cover - defensive; never wedge the worker
132         logger.error(f"Could not compute next-due delay: {e}")
133         return
134     if delay is None:
135         return # nothing waiting
136     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140 def _arm_wake_timer(delay_sec: float) -> None:
141     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142     scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143     monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144     import backend.main as _main

```

```

145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153

```

21. user (2026-06-21T02:34:46.350Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid
6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
33    from backend.results import Failure # standardized error/result contract
34    from backend.reporting import emit_indexer_report
35
36    app = FastAPI(title="Sports Highlight Indexer API")
37
38    # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
39    # rejected by browsers per the fetch spec and is an unsafe default to carry into the
40    # auth/tenancy work; this tool uses no cookies, so credentials stay off.
41    app.add_middleware(
42        CORSMiddleware,
43        allow_origins=["*"],
44        allow_credentials=False,
45        allow_methods=["*"],
46        allow_headers=["*"],
47    )

```

```

48
49 # Serves static frontend files directly (now under api/ per approved structure)
50 os.makedirs("backend/api/static", exist_ok=True)
51 app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
52
53 @app.get("/", response_class=HTMLResponse)
54 def serve_index():
55     index_path = "backend/api/static/index.html"
56     if os.path.exists(index_path):
57         with open(index_path, "r", encoding="utf-8") as f:
58             return f.read()
59     return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
60
61 # Global variables initialized at startup
62 db_instance: Optional[Database] = None
63 global_config: Optional[AppConfig] = None # now typed (Pydantic model)
64
65 # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
66 # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
67 # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
68 # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
69 # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
70 # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
71 # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
72 from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
73     JobWaiting,
74     _arm_wake_timer,
75     _drain_due_jobs,
76     _get_executor,
77     _job_to_request,
78     _MAX_DRAIN_WAKE_SEC,
79     _run_claimed_job,
80     _schedule_next_drain_if_waiting,
81 )
82
83 # Legacy thin wrapper for any remaining direct calls during transition.
84 # New code should import load_app_config / AppConfig from backend.config directly.
85 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
86     cfg = load_app_config(config_path)
87     return cfg.model_dump(mode="json")
88
89 # --- API Models ---
90 class ProcessRequest(BaseModel):
91     video_path: str
92     player_pool: Optional[List[str]] = None
93     max_frames: Optional[int] = None
94     segmenter: Optional[str] = None
95
96 class QueryRequest(BaseModel):
97     video_id: str
98     server: Optional[str] = None
99     receiver: Optional[str] = None
100     duration_min: Optional[float] = None
101     event_type: Optional[str] = None
102
103 class CompileRequest(BaseModel):
104     video_id: str
105     segment_ids: List[int]
106     output_filename: str
107
108 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:

```

```

109         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
110
111         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
112         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
113         and
114         keep the prior good results intact ? a failed/empty re-run never destroys prior
115         data.
116         """
117         if segments:
118             db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
119         else:
120             db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
121
122     # --- API Endpoints ---
123     @app.get("/api/config")
124     def get_config():
125         return global_config
126
127     @app.get("/api/health")
128     def health():
129         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
130         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
131         confirm the engine is reachable and which detector is wired. Never raises."""
132         sport = getattr(global_config, "sport", None)
133         indexing = getattr(global_config, "indexing", None)
134         return {
135             "status": "ok",
136             "sport": sport,
137             "default_segmenter": getattr(indexing, "default_segmenter", None),
138         }
139
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on
144         the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145         the sync endpoint used to return (UI compatibility); the per-video observability
146         report is still emitted in `finally:` and grafted as response["reports"].
147
148         `progress` is an optional callable(stage: str) used to surface coarse job progress.
149         Raises on unexpected internal errors ? the caller records those as a job failure
150         (after this function has already restored the pre-run segments, see below).
151         """

```

22. assistant (2026-06-21T02:34:48.533Z)

Now let me search for the configuration and key contracts around the core engine:

23. user (2026-06-21T02:34:49.560Z)

```

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/base.py:    def
run_predict(self, video_win_path: str, output_win_dir: str,
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/base.py-
video_id: Optional[str] = None) -> Optional[str]:
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/base.py-
"""Run inference and return path to trajectory CSV (or None on failure).
--
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/native_wasb_runner.py:    def run_predict(self,
video_win_path: str, output_win_dir: str,
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/native_wasb_runner.py-        video_id:
Optional[str] = None) -> Optional[str]:
/c/Users/avidu/Projects/badminton-highlight-

```



```

indexer/backend/pipeline/detectors/native_wasb_runner.py:        """Run WASB natively on a
video. Returns the path to the trajectory CSV, or None.
--
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/stub_runner.py:
def run_predict(self, video_win_path: str, output_win_dir: str,
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/stub_runner.py-
video_id: Optional[str] = None) -> Optional[str]:
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/stub_runner.py-
"""Write a deterministic trajectory CSV and return its path (never touches the GPU/WSL)."""
--
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:    def run_predict(self, video_win_path:
str, output_win_dir: str,
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py-        video_id:
Optional[str] = None) -> Optional[str]:
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:        """Run predict.py on a video.
Returns the Windows path to the produced CSV, or None.
--
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:    def trajectory_to_action_windows(
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py-        points: List[TrajectoryPoint],
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py-        fps: float,
--
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py:
def run_predict(self, video_win_path: str, output_win_dir: str,
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py-
video_id: Optional[str] = None) -> Optional[str]:
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py-
"""Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
--
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py:
def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py-
"""
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/stitcher/compiler.py-
Extracts selected segments from a raw video and stitches them together

```

24. assistant (2026-06-21T02:34:51.288Z)

Let me search for hardcoded paths and device-specific assumptions:

25. user (2026-06-21T02:34:51.676Z)

```

/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/tracknet_runner.py:124:        ``output_win_dir`` is a
Windows directory (under the repo's output/) that
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/detectors/wasb_runner.py:121:        # Resolve relative dirs (the
hybrids pass "output/wasb") BEFORE the /mnt
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/gemini.py:17:
"""Per-video chunk staging dir under output/chunks/<video_id>.

```

26. user (2026-06-21T02:34:54.209Z)

```

/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/gemini.py:23:
return os.path.join("output", "chunks", video_id)
/c/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/gemini.py:119:
temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
/c/Users/avidu/Projects/badminton-highlight-

```

```

indexer/backend/pipeline/segmenters/trajectory_hybrid.py:237:         out_dir =
os.path.join("output", self.family)
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/segmenters/trajectory_hybrid.py:313:         handoff_dir =
os.path.join("output", f"chunks_{provider_name}_handoff")
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/segmenters/yolo_hybrid.py:268:         gemini_dir =
os.path.join("output", f"chunks_{provider_name}_handoff")
/c/Users/avidu/Projects/badminton-highlight-
indexer/backend/pipeline/segmenters/yolo_hybrid.py:407:         chunks_dir =
os.path.join("output", "chunks_yolo")

```

27. assistant (2026-06-21T02:34:56.104Z)

Now let me gather information about the config structure:

28. user (2026-06-21T02:34:56.531Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer. Did you mean config?

29. user (2026-06-21T02:34:57.330Z)

```

/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/ABLATION_LAYER.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/ANALYZERS_AND_RECONCILERS.md
/c/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/agent-
a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-eval-
baseline/README.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-logging-
reporting/README.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-scaffolding-
complete/README.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/checkpoints/2026-06-strategy-
foundation/README.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/CODE_AUDIT_AND_TEST_HARDENING-
COMPLETED-2026-06-14.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/COMMERCIAL_READINESS_REVIEW.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/docs/archives/decisions/DECISIONS.md

```

30. user (2026-06-21T02:34:59.353Z)

```

/c/Users/avidu/Projects/badminton-highlight-indexer/docs/ABLATION_LAYER.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/ANALYZERS_AND_RECONCILERS.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/docs/archives/checkpoints/2026-06-14-hardening-loop-complete.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/checkpoints/2026-06-eval-
baseline/README.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/checkpoints/2026-06-logging-
reporting/README.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/docs/archives/checkpoints/2026-06-scaffolding-complete/README.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/checkpoints/2026-06-strategy-
foundation/README.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/CODE_AUDIT_AND_TEST_HARDENING-

```

```
COMPLETED-2026-06-14.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/COMMERCIAL_READINESS_REVIEW.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/decisions/DECISIONS.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/DEFERRED_HARDENING_PLAN-
COMPLETED-2026-06-14.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/HARDENING_LOOP_HANDOFF-
COMPLETED-2026-06-14.md
/c/Users/avidu/Projects/badminton-highlight-indexer/docs/archives/MONETIZATION_AUDIT.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/01-EXISTING-PLANS-AND-GAP.md
/c/Users/avidu/Projects/badminton-highlight-
indexer/docs/archives/past_projects/DECOUPLED_COMPUTE/02-COMPUTE-MAP-AND-CONTRACT.md
```

31. assistant (2026-06-21T02:35:01.681Z)

Now let me compile my comprehensive mapping. I have a clear picture of the core engine. Let me verify the final key architectural seams:

32. user (2026-06-21T02:35:02.313Z)

```
6:class BasePlaySegmenter(ABC):
7-     def __init__(self, db: Database, config: Dict[str, Any]):
8-         self.db = db
9-         self.config = config
10-
11-     @property
12-     --
24:     def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None,
max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25-         """
26-         Processes a video to detect play segments, populates SQLite database,
27-         and returns a Result.
28-
29-         Use isinstance(result, Failure) (or result.is_success()) to distinguish
30-         --
35:class SegmenterRegistry:
36-     def __init__(self):
37-         self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
38-
39-     def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40-         """Registers a new segmenter class under its name property."""
```

33. assistant (2026-06-21T02:35:04.252Z)

Perfect. Now I have all the information I need to create a comprehensive mapping. Let me structure my findings:

34. user (2026-06-21T02:36:10.910Z)

Structured output provided successfully

35. assistant (2026-06-21T02:36:18.322Z)

Perfect! I've completed a comprehensive mapping of the core rally engine. Here's a summary of what I've documented:

Key Findings

****Core Contracts (Behaviour-Identical Across Modes)****

1. ****Detection GPU Compute****: `DetectorRunner.run\_predict()` ? CSV (Frame,Visibility,X,Y). Three interchangeable implementations?WasbRunner (WSL), NativeWasbRunner (Linux GPU), StubDetectorRunner (CPU mock)?all produce identical trajectory schema.

2. **Windowing (Pure CPU Function)**: ``TrackNetRunner.trajectory_to_action_windows()`` is a shared `@staticmethod` across ALL runners. Deterministic, no GPU, no I/O?identical outputs for identical inputs. Critical for regression testing and reuse.
3. **Stitching (FFmpeg + Config)**: ``VideoCompiler.compile_highlights()`` uses pure filesystem operations and ffmpeg subprocess calls. Deterministic given same input; works identically on any box with ffmpeg.

Deployment Mode Abstraction

The single decision point is ``TrajectoryHybridSegmenter._resolve_runner(idx_cfg)``:

- Reads ``RALLY_DETECTOR_IMPL`` env or ``indexing.detector_impl`` config
- Returns the appropriate `DetectorRunner` (stub, native, or WSL-based)
- All downstream code is agnostic to the choice

Critical Coupling Points

Coupling	Issue	Impact
Hardcoded Relative Paths	<code>`'output/'`</code> prefix in segmenters	Breaks on cloud with different CWD
WSL Paths	<code>`~/mnt/c/`</code> translation	Only works on Windows+WSL2
GPU Requirements	WASB/NativeWasb assume NVIDIA GPU	Fails healthcheck on CPU-only boxes
FFmpeg Binary	Assumes ffmpeg/ffprobe in PATH	Fails if tools missing
Scratch Space	Stages to WSL <code>`~/clips`</code> or <code>`output_dir`</code>	Can exhaust disk on large clips
Watermark Fonts	System-dependent fallback	Inconsistent rendering across OSes

The full analysis includes 20 detailed findings with code references, suitable for deployment planning and cloud migration work.